Sentinel Desk — Phase 0: Foundations

Secure agentic RAG platform on Kubernetes · summary of the Phase 0 build and discussion

Repository: github.com/techarx/sentinel-desk · Region: us-east-1 · Prepared 10 November 2026

> **Status.** The cluster side of Phase 0 is complete: `make up` builds the cluster from scratch, `make verify-gvisor` passes, and Argo CD shows `root` and `platform` as Synced / Healthy. The remaining item is a green GitHub Actions run after pushing the CI fix described in section 8.

Contents

1. Project in one paragraph
2. AWS account readiness
3. What Phase 0 built
4. The dev machine (Terraform)
5. How the SSH connection works
6. The Makefile
7. Argo CD and GitOps
8. Problems hit and how they were fixed
9. Why the repo is cloned on the dev machine
10. Automating the bootstrap
11. Daily workflow and cost control
12. Next steps

1. Project in one paragraph

Sentinel Desk is a multi-agent research copilot for a fictional bank's markets desk. It answers questions over SEC filings and a live stream of synthetic trades, and when a question needs computation it writes Python and runs it inside a gVisor (or Kata) sandbox. The point of the project is to show that untrusted, LLM-generated code and adversarial documents can run next to sensitive financial data safely, through layered controls: sandboxing, mTLS, workload identity, fine-grained authorization and guardrails. The plan has nine phases over about ten weeks; Phase 0 lays the foundations everything else builds on.

2. AWS account readiness

The project uses a company-provided AWS lab account. Before building, the account was checked for the limits that most often block a project like this.

Check	Result	Why it matters
Running On-Demand Standard instances (L-1216C47A)	**640 vCPUs**	The default for new accounts is 5, which would block EKS and bare metal. 640 covers the project's peak of about 72 vCPUs.
All Standard Spot Instance Requests (L-34B43A08)	**640 vCPUs**	Separate pool from on-demand; needed if worker or bare-metal nodes use Spot.
Bare metal allowed (m5zn.metal)	**Launched and terminated**	Kata Containers / Firecracker need hardware virtualization. m5zn.metal (48 vCPU) is the smallest current x86 option.
IAM role creation	**Passed**	EKS, node groups, Pod Identity and the dev machine all need IAM roles.
VPCs and internet gateways per region	**Raised from 5 to 7**	The region already had 5 of each, so the project's own VPC could not be created until the limit was raised.

Spot instances in brief

Spot instances are AWS's spare capacity sold at a large discount (often 60–90%), in exchange for AWS being able to reclaim them with two minutes' warning. They suit short, stateless workloads such as sandbox nodes and test sessions. They suit data stores and recorded demos less well. In Terraform, an EKS managed node group becomes Spot with `capacity_type = "SPOT"`; listing several instance types improves the chance of getting capacity.

3. What Phase 0 built

1. A **Linux dev machine in AWS**, created by Terraform, with no open inbound ports.
2. A **secure way into it** from the Mac: SSH tunnelled through AWS Systems Manager (SSM).
3. A **3-node Kubernetes cluster** (kind) on that machine, with one node reserved for sandboxed code.
4. **gVisor** wired into that node, plus a test proving pods there really run inside the sandbox.
5. **Argo CD**, which keeps the cluster in sync with the GitHub repository (GitOps, app-of-apps).
6. **GitHub Actions CI**, a **Makefile** that ties it all together, and **ADR-001** recording the stack decisions.

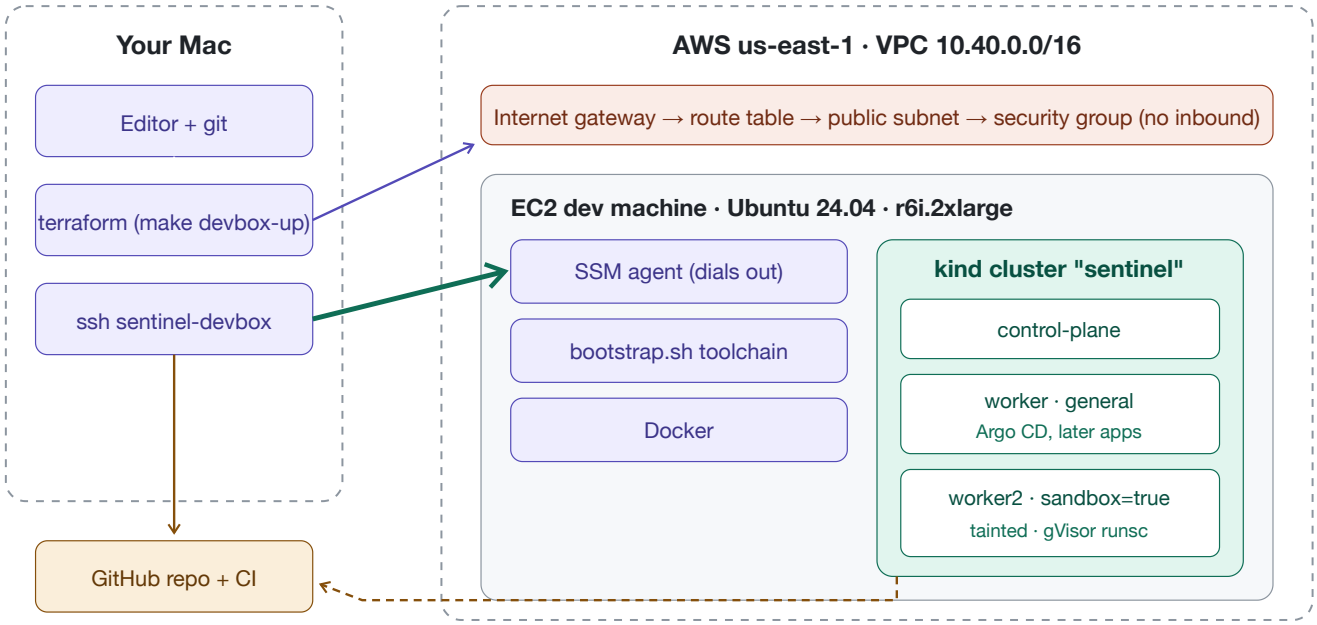


Figure 1. Phase 0 layout. Purple: your tools and the dev machine's services. Teal: Kubernetes. Coral: AWS networking. Amber: GitHub; the dashed line is Argo CD pulling from the repo.

4. The dev machine (Terraform)

Files live in `infra/devbox/`. Terraform describes infrastructure as code; `terraform apply` makes AWS match the description.

Resource	Why it exists
VPC + public subnet	The project's own isolated network; it never touches other teams' VPCs.
Internet gateway + route table	Allows *outbound* traffic: package installs, image pulls and the SSM connection. Without the route, nothing works.

Security group, no inbound rules	Nothing on the internet can open a connection to the machine.
IAM role (AmazonSSMManagedInstanceCore only)	Lets the machine register with SSM and nothing else.
IMDSv2, hop limit 1	Containers on the machine cannot read its AWS credentials from the metadata service (an attack revisited in Phase 4).
Encrypted gp3 disk (150 GB)	Encryption at rest; room for images and data stores.
AWS Budget with alerts	Emails at 50%, 80% and 100% of the monthly limit, plus a forecast alert.
default_tags	Every resource gets `Project=sentinel-desk` for finding and costing it. Activate the tag under Billing → Cost allocation tags so the budget filter matches.

On first boot the machine runs **user data** (`user-data.sh.tftpl`): it installs your SSH public key into `authorized_keys` , then runs `bootstrap.sh` , which installs Docker, gVisor (`runsc`), kind, kubectl, helm and the argocd CLI, logging to `/var/log/sentinel-bootstrap.log` .

5. How the SSH connection works

SSH reaches the dev machine even though port 22 is closed to the world. The trick is that **no connection ever comes in**: both your Mac and the dev machine connect *out* to AWS, and AWS joins the two connections into a tunnel.

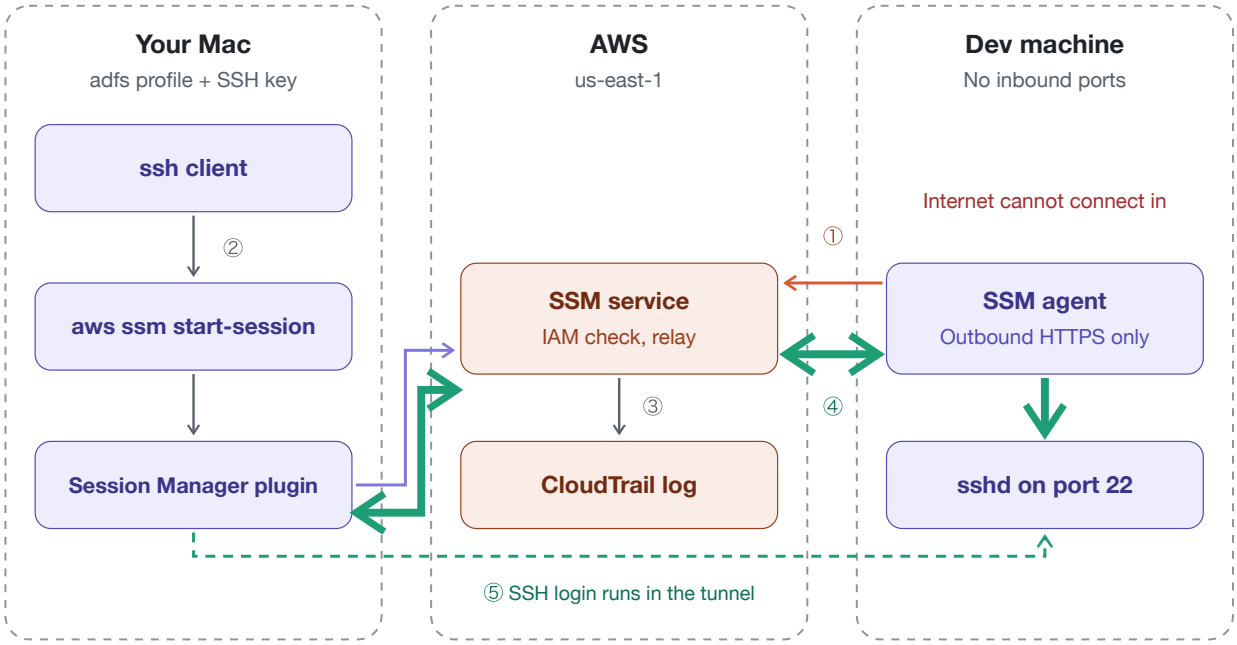


Figure 2. SSH over SSM. Coral arrow: the agent's outbound connection. Thick teal: the joined tunnel. Dashed teal: the SSH session that runs inside it. Numbers match the steps below.

0. **The door is locked.** The security group allows no inbound traffic. Nothing on the internet, including you, can open a connection to the dev machine.
1. **The agent dials out.** At boot, the SSM agent uses the instance's IAM role to open an outbound HTTPS connection to `ssm.us-east-1.amazonaws.com` and keeps it open. This is why the missing internet route caused `TargetNotConnected`: with no route out, AWS had no connection to relay through.
2. **ssh hands off to the AWS CLI.** `ssh sentinel-devbox` reads `~/.ssh/config`. Instead of dialling an IP address, its `ProxyCommand` runs `aws ssm start-session --document-name AWS-StartSSHSession`, and the Session Manager plugin carries the data.
3. **AWS checks who you are.** The request is signed with your `adfs` credentials. SSM checks IAM to decide whether you may open a session to this instance; CloudTrail records who connected, when and to what.
4. **The tunnel is joined.** SSM connects your session to the agent's already-open connection. The agent forwards bytes to `localhost:22` on the machine.
5. **Normal SSH runs inside it.** ssh performs its usual handshake through the tunnel and logs in with your private key, which matches the public key that user data placed in `authorized_keys`.

The SSH config block

```
Host sentinel-devbox
  HostName i-xxxxxxxxxxxxxxxx        # the instance ID, not an IP address
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  ProxyCommand sh -c "aws ssm start-session --region us-east-1 --target %h --document-name
AWS-StartSSHSession --parameters portNumber=%p"
  ServerAliveInterval 60
```

`make devbox-ssh-config` prints this block with the real instance ID. Because the profile is `adfs`, either run `export AWS_PROFILE=adfs` in the terminal first, or add `--profile adfs` after `start-session` in the ProxyCommand.

> **Why this design matters.** Two locks instead of one: valid AWS credentials *and* the SSH key. No open port to scan or attack, and every session is logged. It is the zero-trust approach the whole project is built on, and a strong interview answer to "how do you reach private servers?"

Prerequisites on the Mac

- AWS CLI with working credentials (`adfs` profile).
- Session Manager plugin: `brew install --cask session-manager-plugin`, or AWS's signed installer. Check with `session-manager-plugin`.
- An SSH key pair (`ssh-keygen -t ed25519`); the public key goes in `terraform.tfvars`.

6. The Makefile

`make` runs named groups of commands called **targets**. `make up` runs whatever is defined under `up:`.

Why it matters

- **One command instead of fifteen.** It replaces a long sequence of kind, kubectl and helm commands that must run in the right order.
- **Reproducible.** The same command builds the same cluster every time, which is Phase 0's "done when" criterion.
- **Documentation that cannot go stale.** The Makefile is the instructions; `make help` lists every target from the `##` comments.
- **Shared vocabulary.** The README, you, Claude and later CI all use the same commands.

How make up works

```
up: cluster gvisor platform argocd
```

Target	What it does
`cluster`	Creates the kind cluster from `deploy/kind/cluster.yaml`. Skips if the cluster already exists, so re-running is safe.
`gvisor`	Runs `install-gvisor.sh`: registers `runsc` with containerd on the sandbox node (detecting containerd 1.x vs 2.x config keys) and restarts containerd.
`platform`	`kubectl apply -f deploy/platform/`: namespaces (Pod Security "restricted") and the `gvisor` RuntimeClass, so the cluster is usable before Argo CD syncs.
`argocd`	Installs Argo CD with `helm upgrade --install` (create or update), then applies the root app.

All targets

Where	Target	Purpose
Mac	`devbox-up` / `devbox-down`	Create or destroy the dev machine, VPC and budget.
Mac	`devbox-stop` / `devbox-start`	Stop compute billing between sessions; start again.
Mac	`devbox-ssh-config`	Print the SSH config block.
Dev machine	`up` / `down`	Build or delete the whole cluster.
Dev machine	`verify-gvisor`	Run a test pod under gVisor and check it sees gVisor's kernel, not the host's.
Dev machine	`status`, `argocd-ui`	Show nodes, RuntimeClass and apps; port-forward the Argo CD UI.
Either	`set-repo`, `lint`, `help`	Point Argo CD at the repo; run CI checks locally; list targets.

Details worth knowing

- **`$$` instead of `$`.** make uses `$` for its own variables, so shell variables in recipes are written `$$var`.
- **`.SHELLFLAGS := -eo pipefail -c`.** Stop at the first failing command, including failures inside pipes. `-u` was removed because over SSH bash reads `/etc/bash.bashrc`, which references the unset `PS1`.
- **Idempotent.** Most targets check before acting, so re-running `make up` repairs drift instead of breaking things.

7. Argo CD and GitOps

GitOps means Git is the source of truth for the cluster. Changes are committed, not applied by hand; a controller inside the cluster applies them.

- The **root app** (`deploy/argocd/root-app.yaml`) watches `deploy/argocd/apps/`. Every file there is another Argo CD app.
- Today there is one child, **platform**, which watches `deploy/platform/`.
- To add a component later (Postgres in Phase 1, Istio in Phase 5), add one file to `deploy/argocd/apps/` and push.
- `selfHeal: true` reverts manual changes to match Git; `prune: true` removes resources whose files were deleted.
- "Synced / Healthy" on both apps means the cluster matches Git exactly and everything is running.

8. Problems hit and how they were fixed

Symptom	Cause	Fix
`TargetNotConnected` when SSHing; bootstrap failed	The Terraform had been pointed at the existing MWAAEnvironment VPC. Its only route table had just the local route, so the instance had no path to the internet: the SSM agent timed out reaching AWS and package installs failed.	Raised the VPC and internet gateway limits to 7 and restored the project's own VPC, subnet, internet gateway and route table. The MWAA VPC was left untouched.
`make: No rule to make target`	make was run outside the project folder.	Run make from `~/code/projects/sentinel-desk` (Mac) or `~/sentinel-desk` (dev machine).
gVisor test pod stuck in ContainerCreating	Recent runsc releases need helper binaries (`gvisor_sentry` and others) in a `gvisor-bin/` folder beside runsc; only runsc and the shim had been mounted into the kind node.	Added `/usr/bin/gvisor-bin` as an extra mount in `cluster.yaml` and a check in `install-gvisor.sh`; recreated the cluster.
Test reported FAIL although the pod ran	The check looked for gVisor's old kernel string `4.4.0`; this release reports `4.19.0-gvisor`.	The check now passes when the pod reports a gVisor kernel different from the host's (host: `7.0.0-1014-aws`).

`PS1: unbound variable` noise	The strict `-u` shell flag combined with bash reading `/etc/bash.bashrc` over SSH.	Removed `-u` from the Makefile's shell flags.
CI failed before any job started	`hashFiles()` was used in a job-level `if:`; GitHub only allows it at step level, so it rejected the workflow file.	Moved the condition onto the ruff step. Needs a push to confirm a green run.

> **Mac vs dev machine.** All code edits happen on the Mac; the dev machine gets them through `git pull`. `git checkout -- .` discards uncommitted changes, so only run it on the dev machine, never in the Mac's project folder. A prompt of `mazeez@…` is the Mac; `ubuntu@ip-10-40-…` is the dev machine.

9. Why the repo is cloned on the dev machine

Argo CD exists only *after* the cluster does, so something has to build the cluster first. This is the classic **bootstrap problem**: GitOps cannot install the thing that does GitOps. `make up` needs these files locally where kind runs:

- `deploy/kind/cluster.yaml` to create the cluster;
- `deploy/kind/install-gvisor.sh` to set up gVisor;
- `deploy/platform/*.yaml` for the first apply;
- `deploy/argocd/values.yaml` and `root-app.yaml` to install Argo CD and hand over to it.

After the hand-over, Argo CD pulls everything else from GitHub itself. The clone is needed only for bootstrapping and running make commands.

10. Automating the bootstrap

Level	How	Result
1. Remote targets	Makefile targets such as `make remote-up` that SSH in and run `git pull && make up`.	No manual login; you still trigger it.
2. First-boot bootstrap (recommended next)	Extend user data: after `bootstrap.sh`, clone the repo as `ubuntu` and run `make up && make verify-gvisor`.	`make devbox-up` alone yields a working cluster with Argo CD. Needs a public repo or a read-only GitHub token in Secrets Manager.
3. Target state (Phase 8, EKS)	Terraform creates EKS, then its Helm provider installs Argo CD and applies the root app.	No dev machine or clone in the loop; the industry-standard pattern.

11. Daily workflow and cost control

1. **Start:** `export AWS_PROFILE=adfs`, then `make devbox-start` on the Mac.
2. **Edit** on the Mac; commit and push.

3. **Apply** on the dev machine: `ssh sentinel-devbox`, `cd ~/sentinel-desk && git pull`, then make targets.
4. **Stop:** `make devbox-stop` at the end of every session (disk only is billed while stopped).

- The budget alert covers everything tagged `Project=sentinel-desk`.
- Bare-metal sessions: scale up, test, and destroy the same day; terminated metal instances can take 5–10 minutes to shut down. Check EC2 → Volumes afterwards for leftovers.
- `make devbox-down` removes the dev machine and its VPC entirely.

12. Next steps

1. Push the CI fix and confirm four green jobs (terraform, shell, manifests, python) under GitHub → Actions. That closes Phase 0.
2. Optionally implement Level 2 automation (first-boot bootstrap) and `remote-*` targets.
3. Phase 1 — data layer and real-time ingestion: Postgres + pgvector (CloudNativePG), OpenSearch, Redis Streams, Neo4j; EDGAR ingestion of 10-Ks for 8–10 banks; synthetic trade stream at about 100 events per second. Done when 50+ filings are searchable and stream lag stays under 5 seconds for an hour.

Data sources for Phase 1

Data	Source
10-K / 10-Q filings	SEC EDGAR: `company_tickers.json` for CIKs, `data.sec.gov/submissions/` for filing lists. Send a User-Agent with contact details; stay under 10 requests per second.
Ground-truth financials	EDGAR XBRL `companyfacts` API (for example net interest income) to check answers in the golden set.
Counterparty entities	GLEIF legal entity data (free), giving real parent/subsidiary links for the Neo4j graph.
Trades, customers, PII	Synthetic, generated with Faker.
Prompt-injection training data (Phase 6)	Hugging Face: deepset/prompt-injections, jackhhao/jailbreak-classification; Microsoft BIPIA for indirect injection; plus 200 hand-written examples.